

7. Write a program to implement Logistic Regression (LR) algorithm in python.

a)

CGPA	Aptitude Score	Communication Score	Placement
9.2	90	88	Placed
8.8	85	82	Placed
8.5	80	79	Placed
7.2	72	70	Not Placed
6.8	68	65	Not Placed
9	92	90	Placed
7.5	75	74	Not Placed
8.6	84	81	Placed
6.5	60	58	Not Placed
8.9	88	86	Placed

PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
df = pd.read_excel("Dataset.xlsx", sheet_name="Dataset_1", nrows=10)
print("Dataset:")
print(df)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
new_sample = [[8.9, 87, 84]]
```

```

new_sample = scaler.transform(new_sample)
prediction = model.predict(new_sample)
print("\nNew Sample Prediction:", encoder.inverse_transform(prediction)[0])

```

OUTPUT:

```

Dataset:
   CGPA  Aptitude Score  Communication Score  Placement
0   9.2             90             88      Placed
1   8.8             85             82      Placed
2   8.5             80             79      Placed
3   7.2             72             70  Not Placed
4   6.8             68             65  Not Placed
5   9.0             92             90      Placed
6   7.5             75             74  Not Placed
7   8.6             84             81      Placed
8   6.5             60             58  Not Placed
9   8.9             88             86      Placed

```

Accuracy: 100.0 %

```

Confusion Matrix:
[[1 0]
 [0 2]]

```

New Sample Prediction: Placed

b)

Income (Lakhs)	Credit Score	Loan Amount (Lakhs)	Loan Approved
8	780	5	Yes
7	760	4	Yes
6	720	6	Yes
4	620	8	No
3	580	10	No
9	800	4	Yes
5	650	7	No
8	770	5	Yes
4	600	9	No
7	740	6	Yes

PROGRAM:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
df = pd.read_excel("Dataset.xlsx", sheet_name="Dataset_2", nrows=10)
print("Dataset:")

```

```

print(df)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
new_sample = [[7, 760, 5]]
new_sample = scaler.transform(new_sample)
prediction = model.predict(new_sample)
print("\nNew Sample Prediction:", encoder.inverse_transform(prediction)[0])

```

OUTPUT:

Dataset:

	Income (Lakhs)	Credit Score	Loan Amount (Lakhs)	Loan Approved
0	8	780	5	Yes
1	7	760	4	Yes
2	6	720	6	Yes
3	4	620	8	No
4	3	580	10	No
5	9	800	4	Yes
6	5	650	7	No
7	8	770	5	Yes
8	4	600	9	No
9	7	740	6	Yes

Accuracy: 100.0 %

Confusion Matrix:

```
[[1 0]
 [0 2]]
```

New Sample Prediction: Yes

c)

Temperature (°C)	Heart Rate (bpm)	Oxygen Level (%)	Disease
39	110	94	Positive
38.5	108	95	Positive
37	78	99	Negative
39.2	115	93	Positive
36.8	75	98	Negative
38.8	112	94	Positive
37.2	80	98	Negative
39.1	114	92	Positive
36.7	74	99	Negative
38.9	111	93	Positive

PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
df = pd.read_excel("Dataset.xlsx", sheet_name="Dataset_3", nrows=10)
print("Dataset:")
print(df)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
new_sample = [[38.5, 108, 95]]
new_sample = scaler.transform(new_sample)
prediction = model.predict(new_sample)
print("\nNew Sample Prediction:", encoder.inverse_transform(prediction)[0])
```

OUTPUT:

Dataset:

	Temperature (°C)	Heart Rate (bpm)	Oxygen Level (%)	Disease
0	39.0	110	94	Positive
1	38.5	108	95	Positive
2	37.0	78	99	Negative
3	39.2	115	93	Positive
4	36.8	75	98	Negative
5	38.8	112	94	Positive
6	37.2	80	98	Negative
7	39.1	114	92	Positive
8	36.7	74	99	Negative
9	38.9	111	93	Positive

Accuracy: 100.0 %

Confusion Matrix:

```
[[1 0]
 [0 2]]
```

New Sample Prediction: Positive

d)

Experience	Performance Score	Training Hours	Promotion
10	95	50	Yes
8	90	45	Yes
7	88	40	Yes
3	65	20	No
2	60	18	No
9	93	48	Yes
4	70	25	No
8	89	42	Yes
2	58	15	No
6	85	38	Yes

PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
df = pd.read_excel("Dataset.xlsx", sheet_name="Dataset_4", nrows=10)
print("Dataset:")
print(df)
X = df.iloc[:, :-1]
```

```

y = df.iloc[:, -1]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
new_sample = [[9, 92, 48]]
new_sample = scaler.transform(new_sample)
prediction = model.predict(new_sample)
print("\nNew Sample Prediction:", encoder.inverse_transform(prediction)[0])

```

OUTPUT:

Dataset:

	Experience	Performance Score	Training Hours	Promotion
0	10	95	50	Yes
1	8	90	45	Yes
2	7	88	40	Yes
3	3	65	20	No
4	2	60	18	No
5	9	93	48	Yes
6	4	70	25	No
7	8	89	42	Yes
8	2	58	15	No
9	6	85	38	Yes

Accuracy: 100.0 %

Confusion Matrix:

```

[[1 0]
 [0 2]]

```

New Sample Prediction: Yes

e)

Weight	Diameter	Sweetness	Fruit
150	7.5	90	Apple
160	7.8	88	Apple
140	7.2	91	Apple
120	5.5	70	Orange
125	5.8	72	Orange
130	6	74	Orange
155	7.6	89	Apple
118	5.4	69	Orange
148	7.4	92	Apple
122	5.7	71	Orange

PROGRAM:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_excel("Dataset.xlsx", sheet_name="Dataset_5", nrows=10)

print("Dataset:")

print(df)

X = df.iloc[:, :-1]

y = df.iloc[:, -1]

encoder = LabelEncoder()

y = encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

```
print("\nConfusion Matrix:")  
print(confusion_matrix(y_test, y_pred))  
new_sample = [[155, 7.6, 89]]  
new_sample = scaler.transform(new_sample)  
prediction = model.predict(new_sample)  
print("\nNew Sample Prediction:", encoder.inverse_transform(prediction)[0])
```

OUTPUT:

Dataset:

	Weight	Diameter	Sweetness	Fruit
0	150	7.5	90	Apple
1	160	7.8	88	Apple
2	140	7.2	91	Apple
3	120	5.5	70	Orange
4	125	5.8	72	Orange
5	130	6.0	74	Orange
6	155	7.6	89	Apple
7	118	5.4	69	Orange
8	148	7.4	92	Apple
9	122	5.7	71	Orange

Accuracy: 100.0 %

Confusion Matrix:

```
[[2 0]  
 [0 1]]
```

New Sample Prediction: Apple